

Sequential Ordering Problem: formulación, aplicaciones, estado del arte y análisis crítico de LKH-3

Rodrigo Alejandro Molina Castillo, Bryan Felipe Madrid Campos, Thomas Benjamín Gamonal Lagos

Facultad de Ingeniería, Universidad Andrés Bello

CINF105 – Optimización, Sección 202610.7924

Santiago, Chile

{r.molinacastillo,b.madridcampos,t.gamonallagos}@uandresbello.edu

Resumen—El *Sequential Ordering Problem* (SOP) busca una secuencia de costo mínimo que visite cada nodo exactamente una vez, comience y termine en nodos predefinidos y respete restricciones de precedencia. La combinación de costos de transición dirigidos y órdenes parciales origina un problema NP-hard con aplicaciones en manufactura flexible, talleres de pintura automotriz, optimización de compiladores y planificación de transporte. Este artículo presenta una definición formal, una formulación de programación lineal entera mixta y una revisión crítica de métodos exactos, heurísticos, metaheurísticos y enfoques guiados por aprendizaje. La evidencia sobre SOPLIB se analiza distinguiendo óptimos certificados, mejores soluciones conocidas y resultados heurísticos. Se estudia especialmente LKH-3, considerando la transformación de Jonker-Volgenant para instancias asimétricas, la construcción topológica de una solución inicial factible y la evaluación de precedencias mediante penalización. Los resultados publicados posicionan a LKH-3 como una referencia práctica competitiva para instancias grandes, aunque no garantiza optimalidad. La evidencia directa es más sólida para ACO, algoritmos genéticos, PSO y recocido simulado híbrido; búsqueda tabú, evolución diferencial y GRASP requieren validación sistemática. Persisten desafíos en reproducibilidad, precedencias de baja densidad, benchmarks industriales y evaluación directa de métodos aprendidos sobre SOP.

Palabras clave—Sequential Ordering Problem, restricciones de precedencia, optimización combinatoria, programación entera mixta, metaheurísticas, LKH-3.

Abstract—The Sequential Ordering Problem (SOP) seeks a minimum-cost sequence that visits each node exactly once, starts and ends at predefined nodes, and satisfies precedence constraints. Directed transition costs and partial orders make the problem NP-hard, with applications in flexible manufacturing, automotive paint shops, compiler optimization, and transportation planning. This paper presents a formal definition, a mixed-integer linear programming formulation, and a critical review of exact, heuristic, metaheuristic, and learning-guided approaches. Evidence from SOPLIB is analyzed by distinguishing certified optima, best-known solutions, and heuristic results. Particular attention is given to LKH-3, including the Jonker-Volgenant transformation used for asymmetric instances, topological construction of an initial feasible sequence, and precedence evaluation through a penalty function. Published results position LKH-3 as a competitive practical reference for large instances, although it does not guarantee optimality and comparisons remain sensitive to computational budgets. Direct SOP evidence is strongest for ant colony optimization, genetic algorithms, particle swarm optimization, and hybrid simulated annealing, whereas tabu

search, differential evolution, and GRASP still require systematic validation. Open challenges include reproducibility, low-density precedence structures, industrial benchmarks, and direct evaluation of learning-guided methods on SOP.

Index Terms—Sequential Ordering Problem, precedence constraints, combinatorial optimization, mixed-integer programming, metaheuristics, LKH-3.

I. INTRODUCCIÓN

En numerosos procesos industriales y computacionales no basta con seleccionar un conjunto de tareas: también debe decidirse un orden que minimice costos de transición y respete dependencias obligatorias. El *Sequential Ordering Problem* modela esta situación como la búsqueda de un camino hamiltoniano dirigido de costo mínimo sujeto a precedencias. Fue formulado inicialmente por Escudero [1] y posteriormente estudiado mediante planos de corte, relajación lagrangiana, ramificación y acotamiento, diagramas de decisión, búsqueda local y metaheurísticas [2], [3], [9], [11], [14], [15].

La dificultad no proviene únicamente del crecimiento factorial del espacio de secuencias. Las precedencias convierten muchos movimientos comunes sobre permutaciones en operaciones potencialmente infactibles; por ello, un método eficaz debe optimizar el costo y, simultáneamente, preservar o recuperar la factibilidad. Este trabajo persigue cuatro objetivos: definir y formular matemáticamente el SOP; justificar su complejidad y relevancia práctica; comparar las principales familias de resolución; y posicionar críticamente a LKH-3 dentro del estado del arte. El énfasis es bibliográfico: no se ejecuta el solver, sino que se analiza evidencia publicada y código fuente del tratamiento específico del SOP.

II. DEFINICIÓN FORMAL

Sea $G = (V, A)$ un grafo dirigido, donde $V = \{1, \dots, n\}$ es el conjunto de nodos y $A \subseteq \{(i, j) \in V \times V : i \neq j\}$ contiene las transiciones permitidas. Cada arco $(i, j) \in A$ tiene costo $c_{ij} \geq 0$. Se fijan un nodo inicial s y uno final t , $s \neq t$, y un conjunto acíclico de precedencias $P \subseteq V \times V$, representado mediante un grafo dirigido acíclico. El par $(i, j) \in P$ establece que i debe aparecer antes que j ; la clausura transitiva de P define el orden parcial asociado.

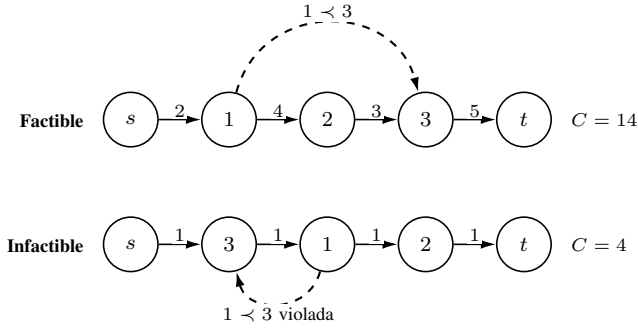


Figura 1. Comparación entre una secuencia factible y una secuencia de menor costo que viola la precedencia $1 < 3$. Las líneas discontinuas representan precedencias y no desplazamientos.

Una solución es una permutación

$$\pi = (\pi_1, \pi_2, \dots, \pi_n), \quad (1)$$

con $\pi_1 = s$, $\pi_n = t$ y cada nodo visitado exactamente una vez. La secuencia es factible si

$$\text{pos}_\pi(i) < \text{pos}_\pi(j), \quad \forall (i, j) \in P. \quad (2)$$

Su costo es

$$C(\pi) = \sum_{k=1}^{n-1} c_{\pi_k, \pi_{k+1}}, \quad (3)$$

y el objetivo es encontrar $\pi^* = \arg \min C(\pi)$ entre las secuencias factibles.

La Fig. 1 diferencia arcos de transición y precedencias. Una ruta puede tener costo bajo y ser inválida si altera el orden parcial. Además, si el grafo de precedencias contiene un ciclo, no existe ninguna secuencia factible.

III. FORMULACIÓN MATEMÁTICA

Se presenta una formulación MILP compacta basada en variables de arco y posición. Para $(i, j) \in A$, $x_{ij} = 1$ si se utiliza la transición $i \rightarrow j$ y 0 en caso contrario. La variable entera u_i representa la posición del nodo i .

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij} \quad (4)$$

$$\text{s.a.} \quad \sum_{j:(i,j) \in A} x_{ij} = 1, \quad \forall i \in V \setminus \{t\}, \quad (5)$$

$$\sum_{j:(t,j) \in A} x_{tj} = 0, \quad (6)$$

$$\sum_{i:(i,j) \in A} x_{ij} = 1, \quad \forall j \in V \setminus \{s\}, \quad (7)$$

$$\sum_{i:(i,s) \in A} x_{is} = 0, \quad (8)$$

$$u_s = 1, \quad u_t = n, \quad (9)$$

$$2 \leq u_i \leq n-1, \quad \forall i \in V \setminus \{s, t\}. \quad (10)$$

$$u_j \geq u_i + 1 - n(1 - x_{ij}), \quad \forall (i, j) \in A, \quad i \neq t, \quad j \neq s. \quad (11)$$

$$u_i + 1 \leq u_j, \quad \forall (i, j) \in P, \quad (12)$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in A, \quad (13)$$

$$u_i \in \mathbb{Z}, \quad \forall i \in V. \quad (14)$$

La función objetivo (4) minimiza el costo. Las restricciones (5)–(8) forman un camino desde s hasta t . Las variables de posición fijan los extremos y, mediante (11), eliminan subciclos desconectados. Las desigualdades (12) imponen el orden parcial, mientras que (13) y (14) declaran explícitamente los dominios binario y entero. La formulación es clara para fines expositivos, pero los métodos exactos competitivos emplean preprocesamiento, separación de desigualdades válidas, planos de corte y cotas específicas [2], [3], [5], [6], [9].

IV. COMPLEJIDAD Y APLICACIONES

Esta sección analiza la complejidad computacional del SOP, el efecto de las restricciones de precedencia y sus principales aplicaciones documentadas.

IV-A. Complejidad computacional

La versión de decisión pregunta si existe una secuencia factible cuyo costo no exceda un umbral K . Dada una secuencia, su costo y el cumplimiento de las precedencias pueden comprobarse en tiempo polinomial.

La versión de decisión pregunta si existe una secuencia factible cuyo costo no exceda un umbral K . Dada una secuencia, su costo y el cumplimiento de las precedencias pueden comprobarse en tiempo polinomial. La NP-dificultad puede justificarse mediante una reducción polinomial desde el problema del camino hamiltoniano dirigido con extremos predefinidos. Dada una instancia (D, s, t) , se conservan los mismos nodos y arcos, se fija $P = \emptyset$, se asigna costo cero a cada arco permitido y se establece $K = 0$. Existe entonces una solución SOP con costo no mayor que K si y solo si D contiene un camino hamiltoniano dirigido desde s hasta t . Por tanto, el SOP es NP-hard [2], [5], [9], [10]. Como la versión de decisión pertenece a NP, se clasifica como NP-completa.

Las precedencias tienen un efecto dual: reducen el número de secuencias admisibles, pero complican la construcción y los movimientos locales. Las colecciones SOPLIB y SOPLIB2006 reúnen instancias sintéticas utilizadas como referencia experimental; en la literatura se estudian casos de 200 a 700 nodos y densidades de precedencia de 1, 15, 30 y 60 % [13], [15], [16]. La búsqueda en árbol de Libralesso *et al.* probó optimalidad en 25 instancias dentro de 600 s y funcionó especialmente bien con densidades de 30 y 60 %; en cambio, las instancias de 1 % permanecieron difíciles por la menor capacidad de poda inducida por precedencias. En consecuencia, el tamaño n no basta para caracterizar la dificultad: también importan la densidad y la estructura del orden parcial.

IV-B. Aplicaciones documentadas

Tabla I
CORRESPONDENCIA ENTRE SOP Y APLICACIONES REALES.

Aplicación	Nodos	Costo	Precedencias
Manufactura flexible	Operaciones de transporte	Tiempo entre operaciones	Dependencias tecnológicas
Pintura automotriz	Vehículos/lotos	Cambio y limpieza de color	Orden impuesto por buffers y líneas
Compilación	Operaciones	Energía de conmutación	Dependencias de datos
Helicópteros	Visitas	Tiempo o distancia	Recogida, entrega y prioridades

La Tabla I sintetiza cuatro aplicaciones rastreables en la literatura. En manufactura flexible, los nodos representan trabajos de transporte y los costos, tiempos de desplazamiento [4]. En talleres de pintura, el objetivo es disminuir limpieza y cambio de color sin alterar el orden impuesto por las líneas de almacenamiento [8]. En compilación, la secuencia de operaciones busca reducir energía de conmutación respetando dependencias de datos [9]. En planificación de helicópteros, las visitas deben satisfacer prioridades y restricciones operacionales [7].

IV-C. SOPLIB e implementación reproducible

SOPLIB2006 se utiliza como benchmark para comparar métodos exactos y heurísticos bajo matrices de costos dirigidos y relaciones de precedencia [6], [14]–[16]. Gouveia y Ruthmair emplearon una variante adaptada de Branch-and-Cut para la versión no capacitada de su problema, equivalente al SOP. En SOPLIB resolvieron por primera vez 12 instancias y mejoraron significativamente las cotas inferiores de las 12 instancias abiertas restantes [6]. Para reproducir resultados no basta con informar el costo: deben conservarse el nombre de la instancia, el archivo original, el límite de tiempo, el hardware, la semilla, el número de ejecuciones y el criterio usado para declarar optimalidad o mejor solución conocida.

Las implementaciones abiertas revisadas muestran dos formatos habituales de entrada. El primero sigue la estructura TSPLIB, con campos como `TYPE: SOP`, `DIMENSION`, `EDGE_WEIGHT_TYPE: EXPLICIT` y `EDGE_WEIGHT_FORMAT: FULL_MATRIX`. El segundo almacena directamente una matriz cuadrada. En los lectores de Skinderowicz y ORProblems, el valor -1 no se interpreta como costo: en la posición (i, j) codifica que j debe preceder a i [18], [19]. El costo de una solución corresponde a los $n - 1$ arcos del camino abierto, sin retorno desde el último nodo al primero.

La validación computacional debe comprobar que cada nodo aparece una sola vez, que se respetan los extremos definidos, que no existen precedencias violadas y que el costo se recalcula desde la matriz. ORProblems implementa un verificador de certificados con estas condiciones, mientras que el repositorio de Skinderowicz incluye lectura de instancias, construcción de soluciones factibles y ejecución automatizada sobre SOPLIB2006 [18], [19]. Adicionalmente, el código de

Libralesso *et al.* permite contrastar la búsqueda en árbol y las soluciones reportadas en su estudio [17]. Estas fuentes de software complementan, pero no sustituyen, los artículos arbitrados que justifican los métodos y resultados.

V. ESTADO DEL ARTE

V-A. Métodos exactos

Escudero introdujo modelos y un procedimiento inexacto para SOP [1]. Ascheuer *et al.* desarrollaron un enfoque de planos de corte aplicado a secuenciación industrial [2], y Escudero *et al.* combinaron relajación lagrangiana con generación de cortes [3]. Posteriormente, Ascheuer, Jünger y Reinelt presentaron una implementación Branch-and-Cut que integra preprocesamiento, separación de desigualdades válidas y ramificación; los autores reportan que instancias reales con más de 200 nodos pudieron resolverse a optimalidad en pocos minutos de CPU [5]. Gouveia y Ruthmair formularon modelos dependientes de carga y precedencias para problemas de recogida y entrega; al eliminar la capacidad, su variante coincide con el SOP, y su Branch-and-Cut resolvió a optimalidad varias instancias que permanecían abiertas en TSPLIB y SOPLIB [6].

Trabajos posteriores incorporaron cotas rápidas, cortes de historia y memorias de estados. Shobaki y Jamal propusieron un algoritmo exacto con aplicación a compiladores [9]; Jamal *et al.* añadieron una cota de asignación y búsqueda local [10]. Los métodos exactos producen certificados de optimalidad, pero su rendimiento depende fuertemente de la estructura de precedencias y de la fortaleza de la relajación lineal.

La búsqueda en árbol de Libralesso *et al.* mostró que una estrategia de Beam Search con equivalencia de prefijos y cotas simples puede superar combinaciones más costosas. En SOPLIB, el método probó optimalidad en aproximadamente la mitad del conjunto y mejoró seis de siete instancias que permanecían abiertas, con un límite de 600 s por instancia; el código de los experimentos fue publicado para facilitar su reproducción [15], [17]. Este resultado cuestiona la idea de que una cota más fuerte siempre mejora el algoritmo: una cota cara puede reducir nodos, pero también ralentizar excesivamente la exploración.

V-B. Heurísticas y metaheurísticas

Las heurísticas constructivas seleccionan únicamente nodos cuyos predecesores ya fueron visitados. Son rápidas, pero una decisión miope puede encarecer el resto de la ruta. La búsqueda local especializada utiliza operadores como SOP-3-exchange, que consideran simultáneamente asimetría y precedencias [11]. Sobre esta base, la literatura permite distinguir las siguientes metaheurísticas.

Colonia de hormigas (ACO). Las hormigas construyen secuencias probabilísticamente a partir de feromonas e información heurística. Para mantener factibilidad, el conjunto candidato debe contener solo nodos cuyos predecesores ya estén programados. Gambardella y Dorigo combinaron Ant Colony System con SOP-3-exchange [11]; posteriormente, Skinderowicz mejoró este esquema y reportó soluciones

mejores o iguales a las conocidas en 41 de 48 instancias, incluyendo diez nuevos récords [14].

Recocido simulado (SA). El recocido simulado acepta ocasionalmente movimientos que empeoran el costo con una probabilidad dependiente de la temperatura, lo que permite escapar de óptimos locales [21]. En SOP, los vecinos deben conservar las precedencias o ser reparados antes de evaluarse. Skinderowicz incorporó esta regla de aceptación como componente híbrido dentro de ACS/EACS y de SOP-3-exchange. Sus experimentos muestran mejoras de calidad en numerosas instancias, aunque con resultados dependientes del tiempo disponible y sin constituir una evaluación de SA como algoritmo independiente [14].

Búsqueda tabú (TS). La búsqueda tabú extiende la búsqueda local mediante memoria adaptativa: los atributos de movimientos recientes se almacenan temporalmente en una lista tabú para impedir retornos inmediatos y reducir ciclos. Un criterio de aspiración puede autorizar un movimiento prohibido cuando mejora la mejor solución conocida [22], [23]. Para el SOP, los intercambios, reinserciones o movimientos de bloques deben preservar las precedencias; alternatively, la factibilidad puede recuperarse mediante reparación o penalización. No se identificó en el corpus revisado una evaluación directa y concluyente de TS sobre SOPLIB, por lo que se presenta como una metaheurística transferible que requiere validación específica bajo presupuestos comparables.

Algoritmo genético (GA). Cada individuo representa una secuencia y evoluciona mediante selección, cruce y mutación. El desafío central es usar operadores que no dupliquen nodos ni violen precedencias. Seo y Moon propusieron una representación de grafo completo y un algoritmo genético híbrido que combina recombinación con mejora local para SOP [12].

Enjambre de partículas (PSO). La PSO clásica opera en espacios continuos; su adaptación discreta representa cada partícula como una permutación y redefine velocidad y movimiento mediante operaciones sobre secuencias. Anghinolfi *et al.* desarrollaron una PSO híbrida con búsqueda local y mecanismos contra estancamiento, obteniendo resultados competitivos sobre benchmarks SOP [13].

Evolución diferencial (DE). La evolución diferencial genera vectores de prueba mediante diferencias entre individuos y conserva el candidato de mejor aptitud [24]. Como su formulación original es continua, una adaptación a SOP requiere una codificación discreta o de claves aleatorias, un decodificador que produzca una permutación y un mecanismo de reparación o penalización de precedencias. No se identificó evidencia experimental directa sobre SOP en el corpus revisado; por tanto, se presenta como una línea transferible de investigación y no como un método consolidado para SOPLIB.

GRASP. GRASP es una metaheurística multiarranque en la que cada iteración combina una fase de construcción voraz aleatorizada con una fase posterior de búsqueda local [25]. Durante la construcción se genera una solución factible incorporando elementos uno a uno. Los candidatos se evalúan mediante una función voraz adaptativa y los mejor valorados

forman una lista restringida de candidatos (RCL), desde la cual se selecciona aleatoriamente el siguiente elemento. La solución construida se somete después a búsqueda local y se conserva la mejor solución obtenida entre todas las iteraciones.

Una posible adaptación al SOP consistiría en formar la RCL únicamente con nodos cuyos predecesores ya hayan sido visitados y ordenarlos según su costo incremental. La búsqueda local posterior debería emplear movimientos que preserven las precedencias o, alternatively, procedimientos de reparación o penalización. Esta adaptación no corresponde a una propuesta evaluada por Feo y Resende, sino a una transferencia conceptual de la estructura general de GRASP al SOP. En el conjunto de fuentes revisadas para este trabajo no se identificó una evaluación experimental directa de GRASP sobre SOPLIB; por ello, se presenta como una alternativa metodológica que requiere validación específica y no como un método consolidado para este benchmark.

En conjunto, ACO, GA y PSO cuentan con evidencia directa de aplicación al SOP. SA aparece principalmente como componente híbrido de ACS/EACS y SOP-3-exchange. En el corpus revisado, TS, DE y GRASP no presentan evaluaciones directas concluyentes sobre SOPLIB, por lo que su desempeño debería estudiarse mediante experimentos controlados con las mismas instancias, límites de tiempo, semillas y criterios de evaluación.

V-C. Aprendizaje y literatura reciente

La literatura posterior a 2020 específicamente dedicada al SOP es menor que en otros problemas de ruteo. Existen, sin embargo, líneas adyacentes útiles: NeuroLKH aprende conjuntos de candidatos para guiar LKH [29]; RL-LKH adapta decisiones de búsqueda mediante refuerzo [30]; Cook *et al.* estudian búsqueda local penalizada bajo restricciones combinadas [28]; y Goutham *et al.* proponen en 2023 una inserción de envolvente convexa que respeta precedencias, superando a vecino más cercano en 97 % de las instancias examinadas bajo su configuración [31]. Estos resultados no deben presentarse como evidencia directa de superioridad sobre SOPLIB, pero sí como tendencias hacia candidatos aprendidos, heurísticas factibles y penalizaciones adaptativas.

VI. LKH-3 APLICADO AL SOP

Lin y Kernighan propusieron movimientos de profundidad variable para reemplazar arcos en una ruta [20]. Sobre esa base, el informe técnico de Helsgaun describe a LKH como un motor de búsqueda local de alto rendimiento y presenta a LKH-3 como su extensión para problemas restringidos [26]. Debido a que el SOP posee costos dirigidos, LKH-3 emplea una transformación de Jonker-Volgenant basada en el desdoblamiento de nodos para obtener una representación simétrica equivalente [26]. Sobre esta representación, el núcleo Lin-Kernighan ejecuta movimientos k -opt, mientras una función específica evalúa las violaciones de precedencia.

El tratamiento del SOP puede verificarse en la distribución fuente de LKH-3 [27]. La rutina `SOP_InitialTour` genera

Tabla II
RESULTADOS Y CARACTERÍSTICAS REPRESENTATIVAS DEL ESTADO DEL ARTE.

Referencia	Enfoque	Datos/escala	Garantía	Resultado o aporte principal
Ascheuer <i>et al.</i> (1993)	Planos de corte	Casos industriales	Sí, al cerrar	Formulación exacta y cortes para precedencias.
Ascheuer <i>et al.</i> (2000)	Branch-and-Cut	Casos reales y TS-PLIB	Sí, al cerrar	Instancias reales con más de 200 nodos resueltas a optimalidad en minutos.
Gouveia y Ruthmair (2015)	Branch-and-Cut	TSPLIB y SOPLIB	Sí, al cerrar	Cerró 12 instancias SOPLIB y mejoró las cotas inferiores de otras 12.
Shobaki y Jamal (2015)	Branch-and-Bound	SOP y compilación	Sí, al cerrar	Cotas rápidas y memoria de estados dominados.
Jamal <i>et al.</i> (2017)	B&B híbrido	SOPLIB	Sí, al cerrar	Cota de asignación y búsqueda local integrada al árbol.
Skinderowicz (2017)	EACS-SA + búsqueda local	48 SOPLIB2006	No	41/48 resultados mejores o iguales; 10 nuevas mejores soluciones.
Helsgaun (2017)	LKH-3 penalizado	163 instancias SOP	No	9 resultados mejores que los valores de referencia, 150 iguales y 4 peores.
Libralesso <i>et al.</i> (2019)	Beam Search + memoria	200–700 nodos	Puede certificar	25 instancias cerradas; 6/7 casos abiertos mejorados en 600 s.
Goutham <i>et al.</i> (2023)	Inserción factible	TSPLIB con precedencias	No	Superó a vecino más cercano en 97 % de los casos evaluados.

una solución inicial mediante ordenamiento topológico: mantiene grados de entrada, incorpora solo nodos sin predecesores pendientes y detecta ciclos de precedencia. Para el SOP, el nodo final se fuerza al final de la secuencia. Posteriormente, `Penalty_SOP` asigna a cada nodo su posición actual y calcula una penalización compuesta por dos elementos: la desviación del nodo final respecto de la última posición y el número de precedencias (i, j) con $\text{pos}(i) > \text{pos}(j)$. La rutina incluye terminación anticipada cuando la penalización ya no puede mejorar la solución actual y revisa primero restricciones asociadas a nodos afectados por intercambios recientes.

En términos conceptuales, LKH-3 compara primero la factibilidad y luego el costo: una penalización menor domina a una mayor y, entre soluciones con la misma penalización, se favorece el menor costo. Esta separación permite reutilizar el núcleo de búsqueda local sin ocultar las restricciones. En el resumen de pruebas del informe técnico se reportan 163 instancias SOP: 9 resultados mejoraron los valores de referencia utilizados, 150 los igualaron y 4 fueron peores [26]. Estos valores no certifican optimalidad ni demuestran superioridad universal, pues la mejor solución conocida puede actualizarse y el resultado depende de los conjuntos candidatos, los reinicios, las semillas, el tiempo y los parámetros.

La comparación publicada por Libralesso *et al.* advierte además que los presupuestos no eran equivalentes: LKH-3 había sido ejecutado durante 100000 s por instancia, mientras que EACS-SA acumulaba 18000 s y la búsqueda en árbol se limitó a 600 s [15]. Por ello, afirmar que un método “vence” a otro únicamente por el costo final sería metodológicamente débil. La contribución distintiva de LKH-3 es su arquitectura reutilizable, su búsqueda k -opt y su verificación incremental de restricciones, no una garantía universal de superioridad.

VII. DISCUSIÓN Y LÍNEAS ABIERTAS

La evidencia permite formular cuatro conclusiones críticas. Primero, la densidad de precedencias modifica la naturaleza

de la dificultad: los métodos de árbol pueden cerrar con rapidez instancias densamente restringidas, mientras que las de baja densidad requieren mejores relajaciones de ruta. Segundo, la calidad debe compararse bajo el mismo tiempo, hardware, número de ejecuciones y semillas. Tercero, los benchmarks clásicos de 200–700 nodos son valiosos, pero no sustituyen datos industriales contemporáneos. Cuarto, los métodos aprendidos deben integrarse con mecanismos que garanticen factibilidad y evaluarse directamente sobre SOP.

Las líneas abiertas incluyen: selección dinámica de candidatos para LKH-3; operadores que preserven precedencias sin reparaciones costosas; modelos híbridos exacto-heurísticos; generación de benchmarks con estructuras de precedencia realistas; y análisis estadístico con múltiples ejecuciones. También conviene caracterizar las instancias mediante densidad, longitud de la cadena crítica y anchura del orden parcial, en lugar de usar únicamente n .

VIII. CONCLUSIONES

El SOP busca una secuencia dirigida de costo mínimo bajo un orden parcial. Es NP-hard y su resolución práctica depende tanto del número de nodos como de la estructura de precedencias. Los métodos exactos permiten certificar optimalidad, mientras que las metaheurísticas y la búsqueda local especializada ofrecen soluciones de alta calidad para instancias mayores. La evidencia directa es sólida para ACO, GA y PSO; SA aparece principalmente como componente híbrido de ACS/EACS y SOP-3-exchange. En cambio, TS, DE y GRASP deben tratarse como líneas de adaptación todavía no validadas de manera concluyente sobre SOPLIB.

LKH-3 ocupa una posición relevante porque combina un motor Lin-Kernighan altamente optimizado con una inicialización topológica y una penalización específica que cuenta violaciones de precedencia. Sin embargo, sigue siendo heurístico y debe evaluarse con presupuestos homogéneos. La principal oportunidad de investigación consiste en combinar candidatos aprendidos,

factibilidad explícita y protocolos reproducibles sobre nuevas instancias SOP con estructuras industriales realistas.

DECLARACIÓN DE USO DE IA

Se utilizó ChatGPT para proponer una estructura preliminar, generar un primer borrador en $\text{\LaTeX}$, apoyar la redacción y sugerir términos y fuentes iniciales de búsqueda. Los autores seleccionaron las referencias conservadas, contrastaron las afirmaciones con los documentos originales y validaron la formulación matemática, el análisis crítico y las conclusiones. Todas las referencias incluidas fueron verificadas y leídas por al menos un integrante del grupo.

REFERENCIAS

- [1] L. F. Escudero, "An inexact algorithm for the sequential ordering problem," *European Journal of Operational Research*, vol. 37, no. 2, pp. 236–249, 1988.
- [2] N. Ascheuer, L. F. Escudero, M. Grötschel, and M. Stoer, "A cutting plane approach to the sequential ordering problem (with applications to job scheduling in manufacturing)," *SIAM Journal on Optimization*, vol. 3, no. 1, pp. 25–42, 1993.
- [3] L. F. Escudero, M. Guignard, and K. Malik, "A Lagrangian relax-and-cut approach for the sequential ordering problem with precedence relationships," *Annals of Operations Research*, vol. 50, no. 1, pp. 219–237, 1994.
- [4] N. Ascheuer, "Hamiltonian path problems in the on-line optimization of flexible manufacturing systems," Konrad-Zuse-Zentrum für Informationstechnik Berlin, Tech. Rep. TR 96-3, 1996.
- [5] N. Ascheuer, M. Jünger, and G. Reinelt, "A Branch & Cut Algorithm for the Asymmetric Traveling Salesman Problem with Precedence Constraints," *Computational Optimization and Applications*, vol. 17, pp. 61–84, 2000.
- [6] L. Gouveia and M. Ruthmair, "Load-dependent and precedence-based models for pickup and delivery problems," *Computers & Operations Research*, vol. 63, pp. 56–71, 2015, doi: 10.1016/j.cor.2015.04.008.
- [7] M. T. Fiala Timlin and W. R. Pulleyblank, "Precedence constrained routing and helicopter scheduling: Heuristic design," *Interfaces*, vol. 22, no. 3, pp. 100–111, 1992, doi: 10.1287/inte.22.3.100.
- [8] S. Spieckermann, K. Gutenschwager, and S. Voß, "A sequential ordering problem in automotive paint shops," *International Journal of Production Research*, vol. 42, no. 9, pp. 1865–1878, 2004, doi: 10.1080/00207540310001646821.
- [9] G. Shobaki and J. Jamal, "An exact algorithm for the sequential ordering problem and its application to switching energy minimization in compilers," *Computational Optimization and Applications*, vol. 61, no. 2, pp. 343–372, 2015, doi: 10.1007/s10589-015-9725-9.
- [10] J. Jamal, G. Shobaki, V. Papapanagiotou, L. M. Gambardella, and R. Montemanni, "Solving the sequential ordering problem using branch and bound," in *Proc. IEEE Symposium Series on Computational Intelligence*, 2017, pp. 1–9.
- [11] L. M. Gambardella and M. Dorigo, "An ant colony system hybridized with a new local search for the sequential ordering problem," *INFORMS Journal on Computing*, vol. 12, no. 3, pp. 237–255, 2000, doi: 10.1287/ijoc.12.3.237.12636.
- [12] D.-I. Seo and B.-R. Moon, "A hybrid genetic algorithm based on complete graph representation for the sequential ordering problem," in *Proc. Genetic and Evolutionary Computation Conference*, 2003, pp. 669–680.
- [13] D. Anghinolfi, R. Montemanni, M. Paolucci, and L. M. Gambardella, "A hybrid particle swarm optimization approach for the sequential ordering problem," *Computers & Operations Research*, vol. 38, no. 7, pp. 1076–1085, 2011, doi: 10.1016/j.cor.2010.10.014.
- [14] R. Skinderowicz, "An improved ant colony system for the sequential ordering problem," *Computers & Operations Research*, vol. 86, pp. 1–17, 2017, doi: 10.1016/j.cor.2017.04.012.
- [15] L. Libralesso, A.-M. Bouhassoun, H. Cambazard, and V. Jost, "Tree search algorithms for the Sequential Ordering Problem," arXiv:1911.12427, 2019, doi: 10.48550/arXiv.1911.12427. [Online]. Available: <https://arxiv.org/abs/1911.12427>.
- [16] IDSIA, "SOPLIB2006: Benchmark instances for the Sequential Ordering Problem," benchmark data set. [Online]. Available: <https://people.idsia.ch/~roberto/SOPLIB06.zip>. Accessed: Jun. 18, 2026.
- [17] L. Libralesso, "cats-ts-sop: Source code of the experiments performed in 'Tree search algorithms for the Sequential Ordering Problem'," GitLab repository, 2019. [Online]. Available: <https://gitlab.com/librallu/cats-ts-sop>. Accessed: Jun. 18, 2026.
- [18] R. Skinderowicz, "AntColonySystemSA: Ant Colony System and Simulated Annealing implementation for the Sequential Ordering Problem," GitHub repository, 2017. [Online]. Available: <https://github.com/RSkinderowicz/AntColonySystemSA>. Accessed: Jun. 18, 2026.
- [19] F. Fontan, "ORProblems: Sequential Ordering Problem data structures, SOPLIB/TSPLIB parsers and certificate checker," GitHub repository. [Online]. Available: https://github.com/fontanf/orproblems/blob/master/include/orproblems/scheduling/sequential_ordering.hpp. Accessed: Jun. 18, 2026.
- [20] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling-salesman problem," *Operations Research*, vol. 21, no. 2, pp. 498–516, 1973, doi: 10.1287/opre.21.2.498.
- [21] S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983, doi: 10.1126/science.220.4598.671.
- [22] F. Glover, "Tabu search—Part I," *ORSA Journal on Computing*, vol. 1, no. 3, pp. 190–206, 1989, doi: 10.1287/ijoc.1.3.190.
- [23] F. Glover, "Tabu search—Part II," *ORSA Journal on Computing*, vol. 2, no. 1, pp. 4–32, 1990, doi: 10.1287/ijoc.2.1.4.
- [24] R. Storn and K. Price, "Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, pp. 341–359, 1997, doi: 10.1023/A:1008202821328.
- [25] T. A. Feo and M. G. C. Resende, "Greedy randomized adaptive search procedures," *Journal of Global Optimization*, vol. 6, pp. 109–133, 1995, doi: 10.1007/BF01096763.
- [26] K. Helsgaun, "An extension of the Lin–Kernighan–Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems," Roskilde University, Roskilde, Denmark, Tech. Rep., 2017. [Online]. Available: <https://webhotel4.ruc.dk/~keld/research/LKH-3/>.
- [27] K. Helsgaun, "LKH-3 source distribution: SOP_InitialTour.c and Penalty_SOP.c," Roskilde University. [Online]. Available: <https://webhotel4.ruc.dk/~keld/research/LKH-3/>. Accessed: Jun. 14, 2026.
- [28] W. Cook, S. Held, and K. Helsgaun, "Constrained local search for last-mile routing," arXiv:2112.15192, revised Sep. 2022. [Online]. Available: <https://arxiv.org/abs/2112.15192>.
- [29] L. Xin, W. Song, Z. Cao, and J. Zhang, "NeuroLKH: Combining deep learning model with Lin–Kernighan–Helsgaun heuristic for solving the traveling salesman problem," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 7472–7483.
- [30] J. Zheng, K. He, J. Zhou, Y. Jin, and C.-M. Li, "Reinforced Lin–Kernighan–Helsgaun algorithms for the traveling salesman problems," arXiv:2207.03876, 2022. [Online]. Available: <https://arxiv.org/abs/2207.03876>.
- [31] M. Goutham, M. Menon, S. Garrow, and S. Stockar, "A convex hull cheapest insertion heuristic for precedence constrained traveling salesperson problems or sequential ordering problems," arXiv:2303.05333, 2023. [Online]. Available: <https://arxiv.org/abs/2303.05333>.